Get online today.

Grow your brand with a unique domain name
build a powerful site with GoDaddy.

GoDaddy.com Shop N

Menu

Cyber Security , Web Security

Information disclosure vulnerabilities

April 15, 2023 No Comments

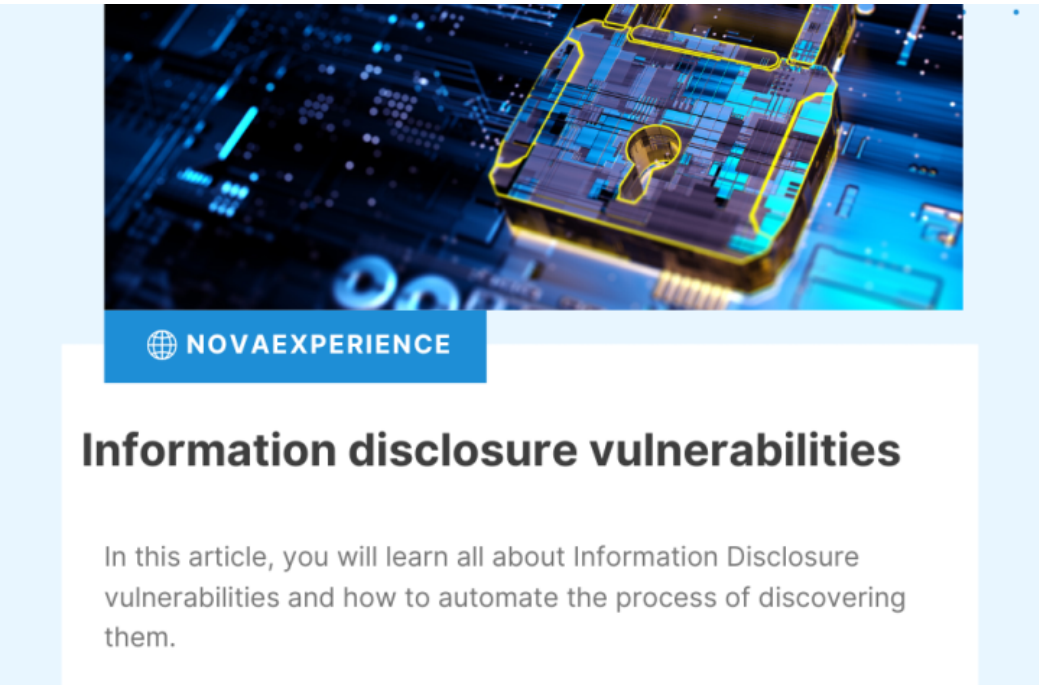


Table of Contents

1. Information disclosure in error messages
2. Information disclosure on debug page
3. Source Code Disclosure via Backup Files

4. Authentication bypass via information disclosure
5. Information disclosure in version control history
6. Automate Information disclosure vulnerabilities

Information disclosure vulnerabilities or Information Leakage is a web vulnerability we have, in which personal information on our website is exposed on the internet. This information could be users' details, website-related files, or data that are not accessible to an unauthorized user, but due to Information Disclosure, it falls into the hands of an attacker who can misuse it.

If we look at Information Disclosure at a **basic level**, we can learn how a website handles sensitive information and how unauthorized users can access that information. At the basic level, we read the website's source code for Information Disclosure vulnerability and try to find secret information in the website's URL.

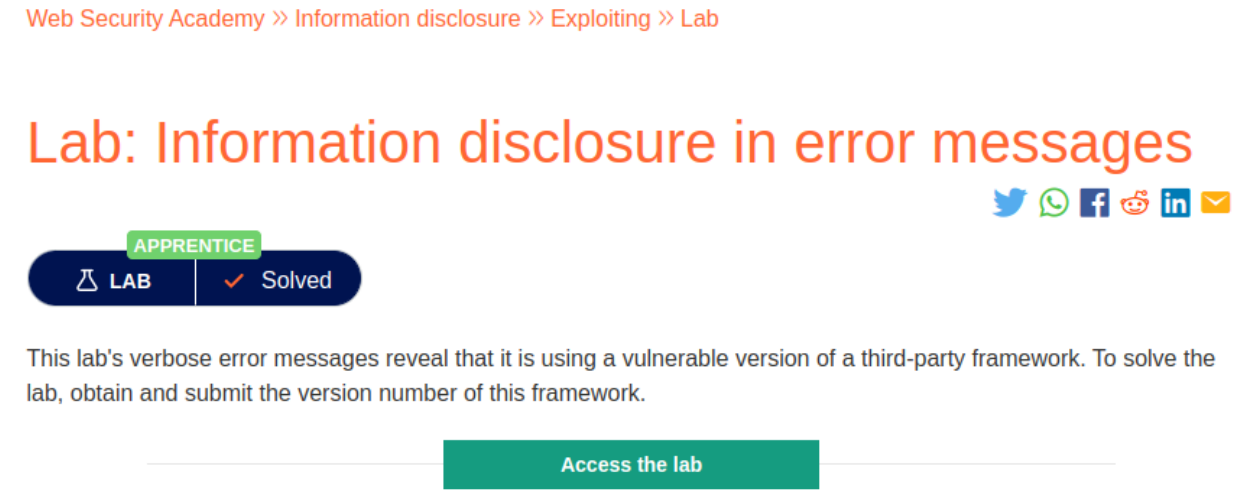
Earn $10 Just by Signing Up - Daily Tasks for Free Cash Back on All Your Favorite Stores

At the **intermediate level**, we explain to a web application pen-tester or security researcher the technical details and concepts of Information Disclosure vulnerabilities. For example, the website's log files, configuration files, or backend database. At this level, a security researcher is taught more technical aspects such as SQL injection, directory traversal, or file inclusion vulnerabilities to achieve Information Disclosure vulnerability.

If we study Information Disclosure vulnerabilities at an **advanced level**, we have to understand advanced concepts or exploitation techniques, such as bypassing techniques. For example, how we can bypass a website's APIs, authentication processes, encryption schemes, and session management to access sensitive data.

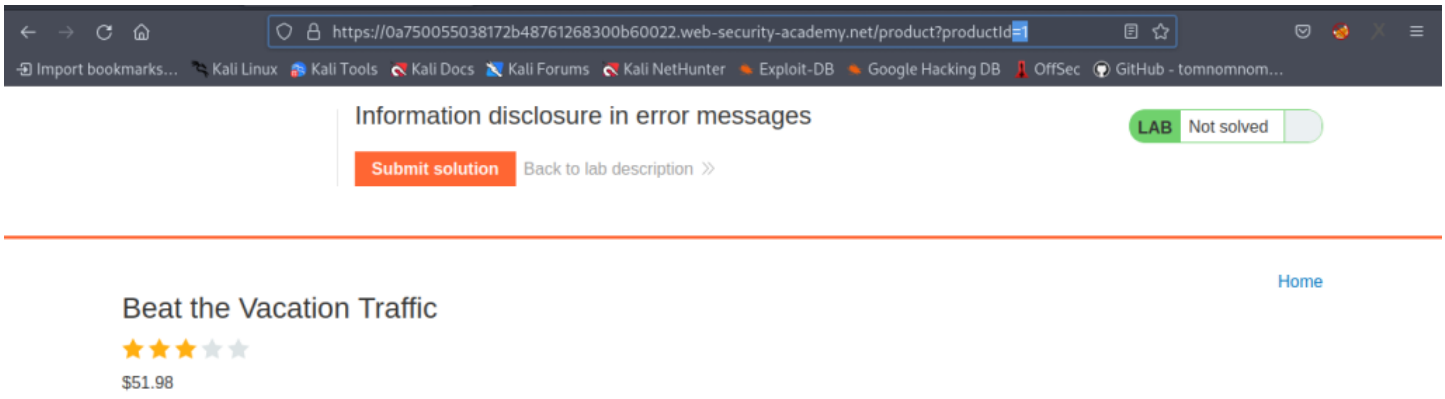
We have various types of Information Disclosure vulnerabilities, such as Information Disclosure in error messages, Information Disclosure on debug pages, Source code disclosure via backup files, Authentication bypass via Information Disclosure, and Information Disclosure in version control history. We will explore each one of them and solve them in practical labs.

Information disclosure in error messages



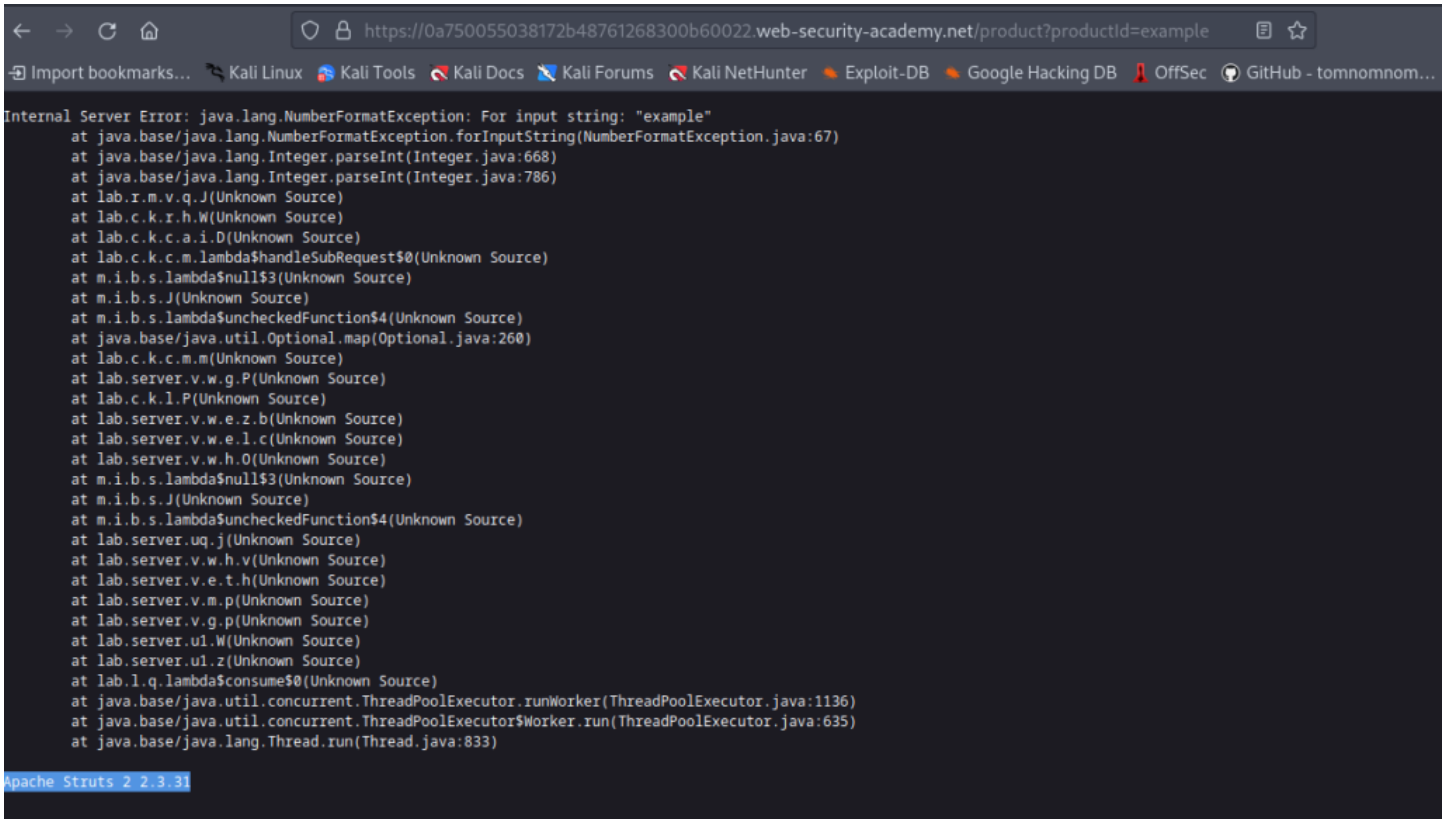
The first lab we have is Information Disclosure in error messages. This lab contains an error message that exposes a vulnerable third-party framework. To solve the lab, we need to find the vulnerable version of that framework.

First, we will access the lab. After accessing the lab, we will click on "view details" of any random product. We know that this vulnerability arises due to error messages.



If we focus on the URL, we have a value being passed in the productId, which is equal to 1 (product?productId=1).

Here, we have written a simple example with 1 to test what response data the application gives us.

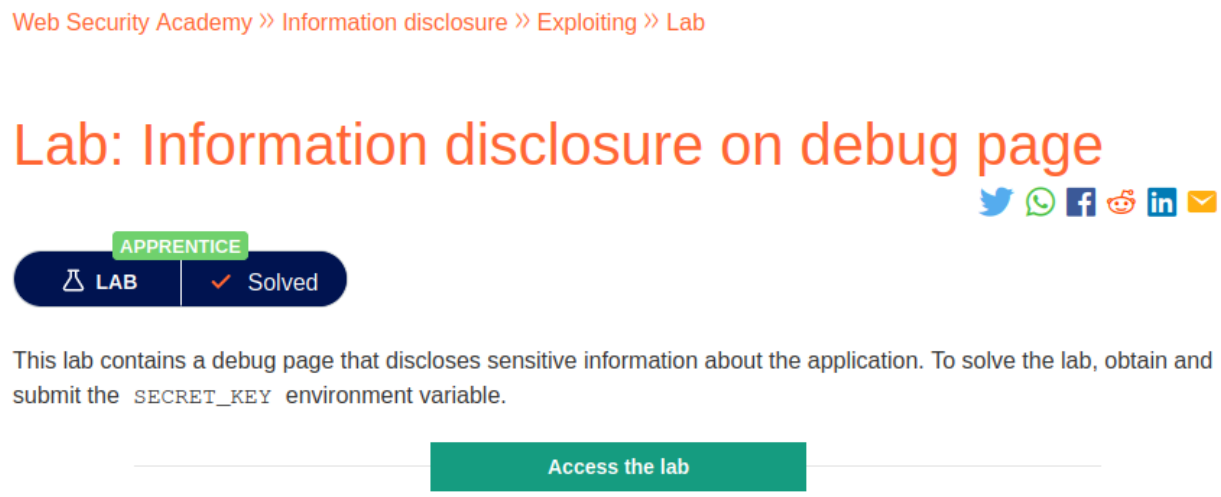


When I submitted the simple example as a parameter, it showed me the vulnerable version of Apache Struts 2 2.3.31. This was our objective to solve this lab. Now, we can simply submit this and the lab will be solved.



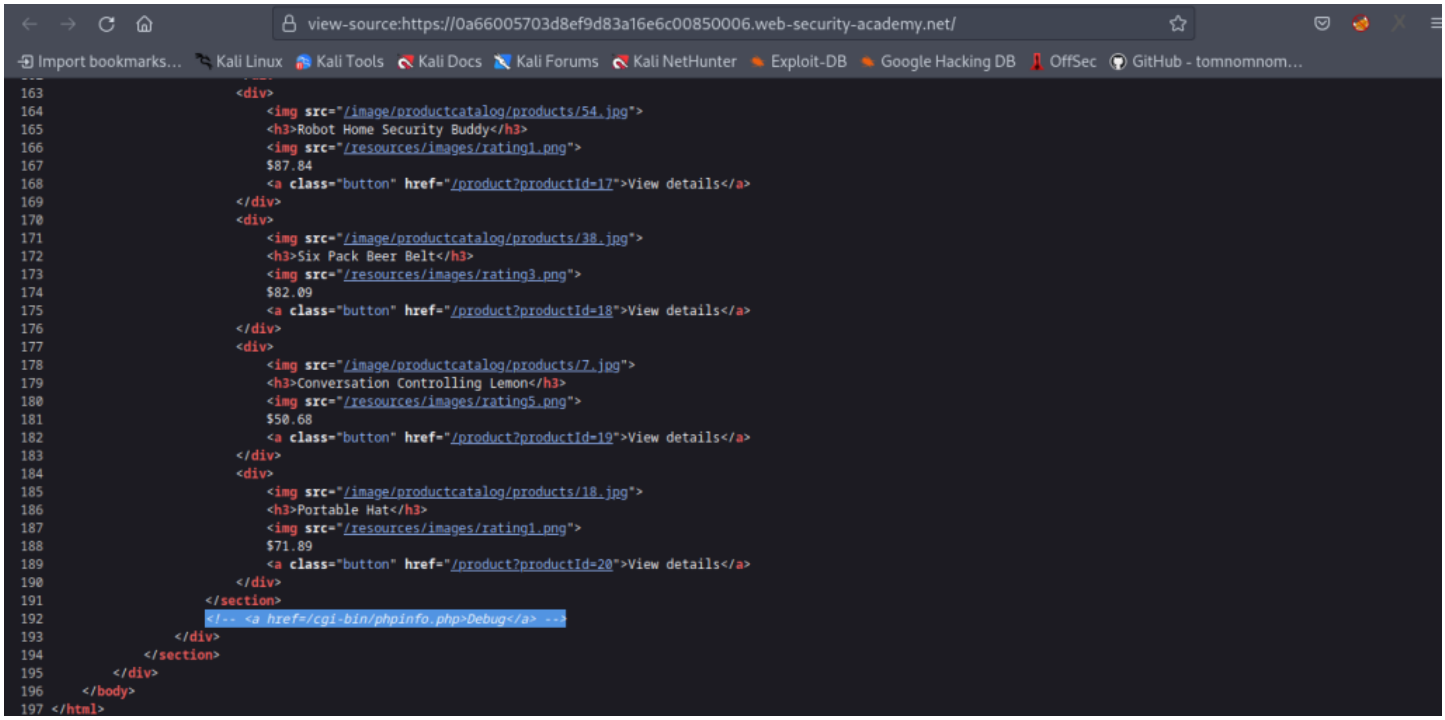
So, we successfully solved our first lab. Now, let's move on to our second lab.

Information disclosure on debug page



In this lab, we have been told that there is a debug page that is disclosing sensitive information. To solve the lab, we need to find a SECRET_KEY in the environment variable.

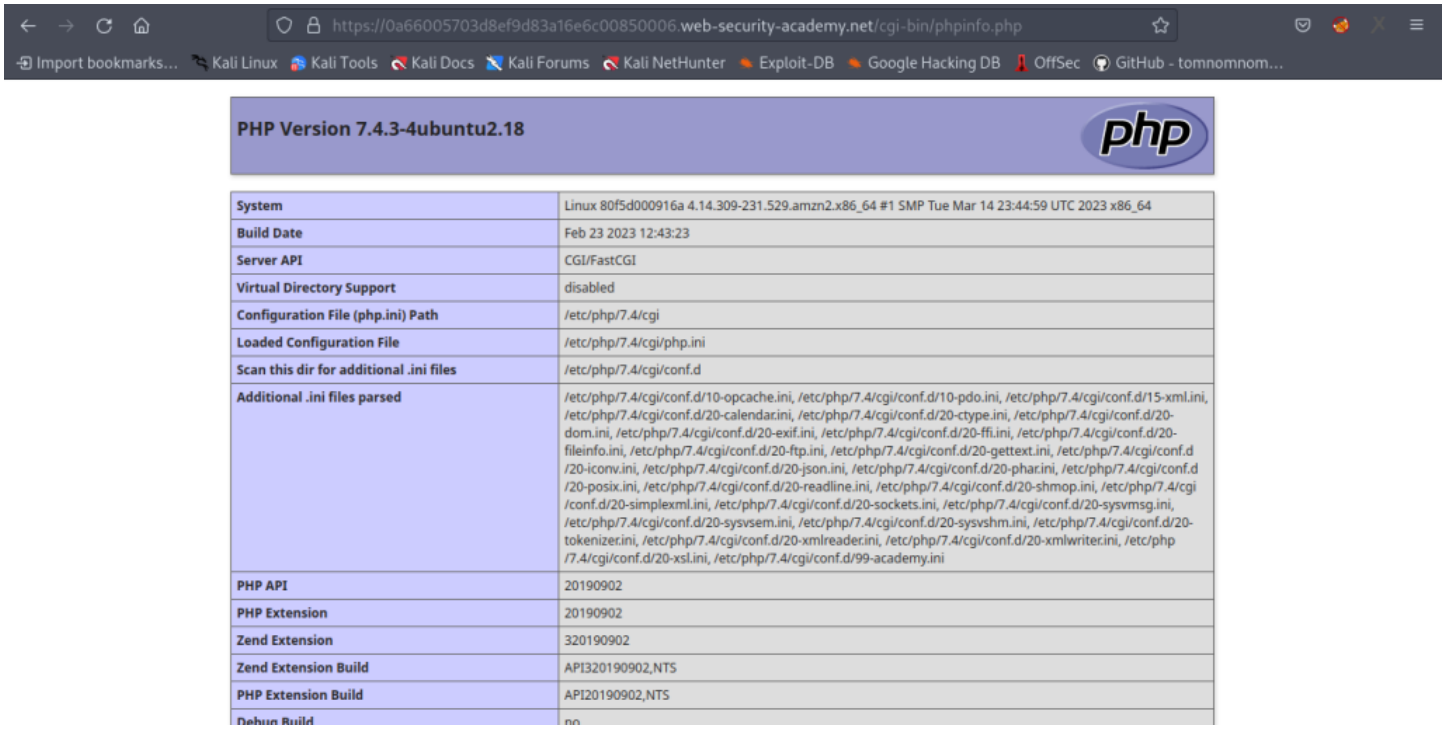
So first of all, we will access the lab.



After accessing the lab, the first thing we did was review the source code. Reviewing the source code helped us find a location.

```
<!-- <a href=/cgi-bin/phpinfo.php>Debug</a> -->
```

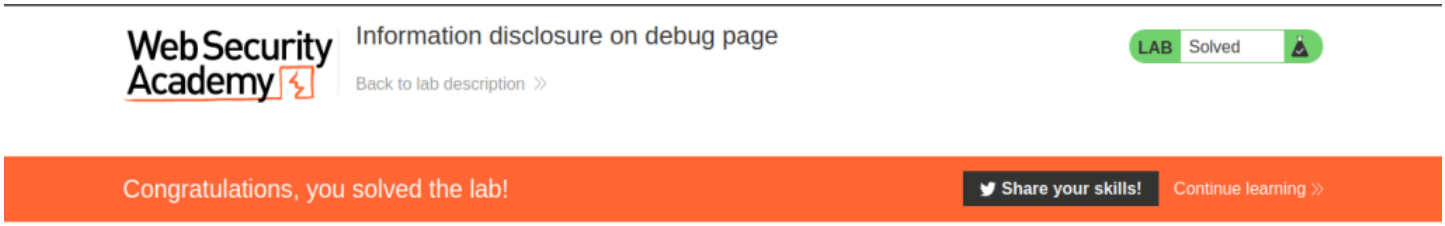
So i found a file in the source code containing PHP info. I added this location to the lab's URL and checked the result.



I appended /cgi-bin/phpinfo.php to the URL of the lab and was redirected to the phpinfo page. Now I hope that the SECRET_KEY we are looking for will be available here.

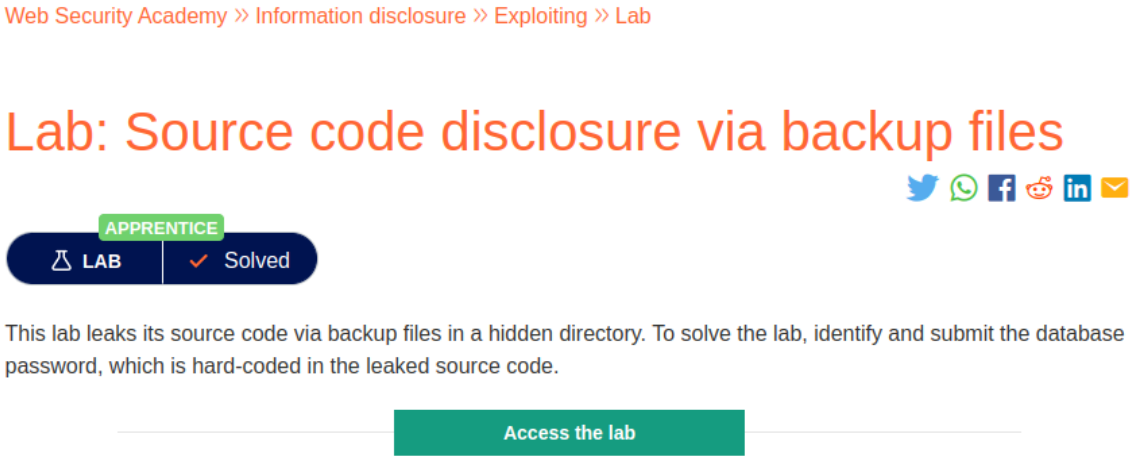
REMOTE_HOST	223.123.88.195
USER	carlos
SECRET_KEY	44jgc5ibjrlb7pmh3inffghzuc4851g8
HTTP_TE	trailers

I have found the SECRET_KEY that I was looking for. Now, let's submit it in the lab and see if it solves our lab or not.



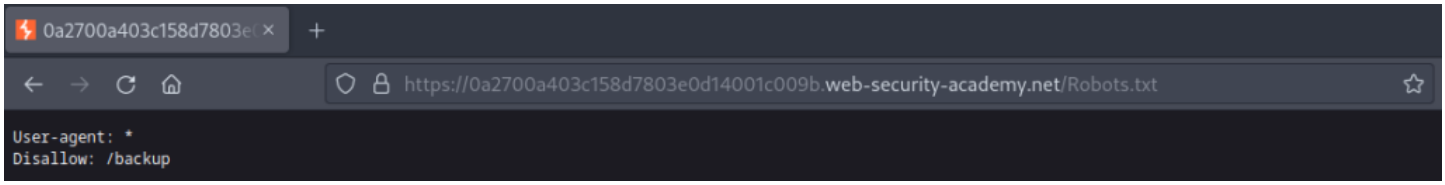
We have successfully solved our 2nd lab which was related to Information Disclosure vulnerabilities. Now, we will move on to our 3rd lab which is about Source Code Disclosure via Backup Files.

Source Code Disclosure via Backup Files

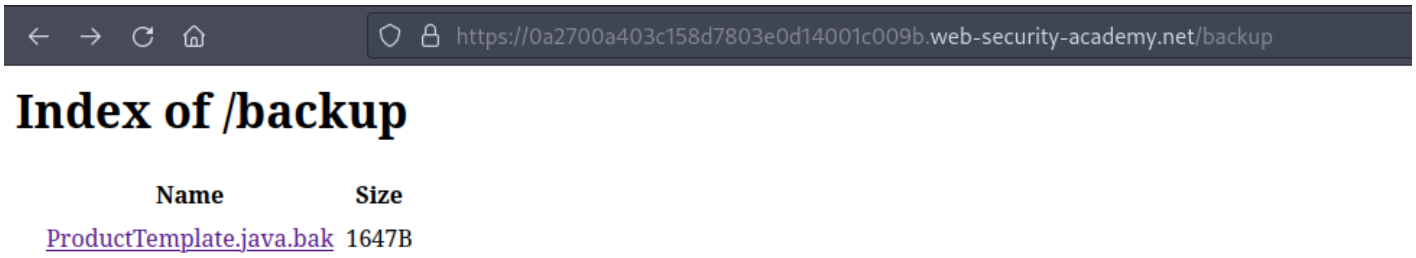


In this lab, we have been given some instructions which say that this lab is leaking source code through some hidden directory backup files. To solve this lab, you need to first locate the database password file that is hard-coded in the form and is also present in the leaked source code.

Firstly, we will access the lab. After accessing the lab, we will check the /Robots.txt file.



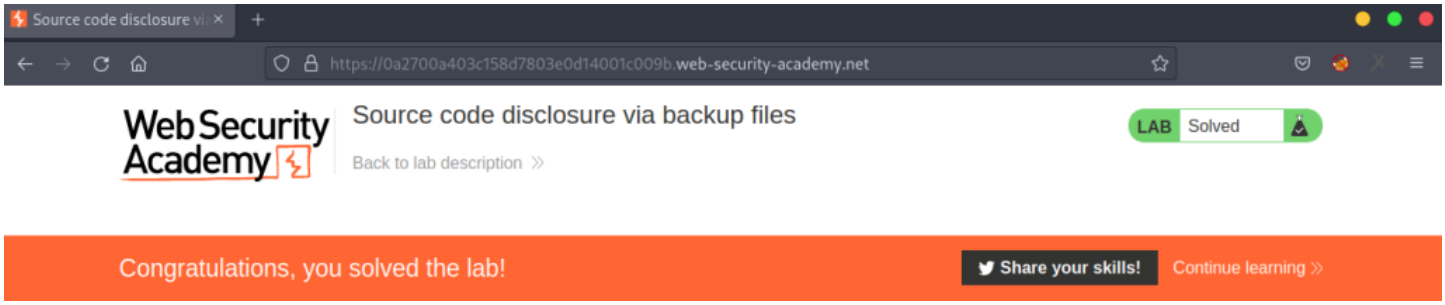
When I viewed the /Robots.txt file, I found a file named “Disallow: /backup” listed there. Now, I will copy the location of this file and paste it in the URL to see what gets downloaded.



I checked out the file named ProductTemplate.java.bak which was 1647B in size, and when I clicked on this file, the backup code of the site was revealed to me.

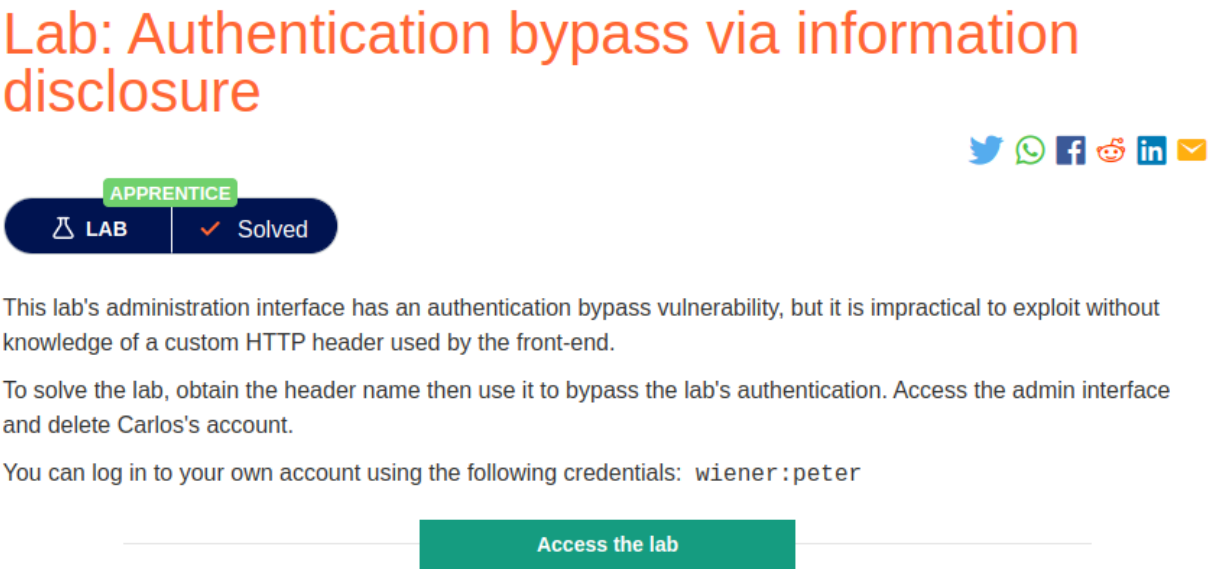


In that backup file, we had the database credentials available, which were hard-coded in the form, but we found them out. Now, we will submit (ataktllf7wr9on52osgkgkcxijdbk4ha) and see if our lab is solved or not.



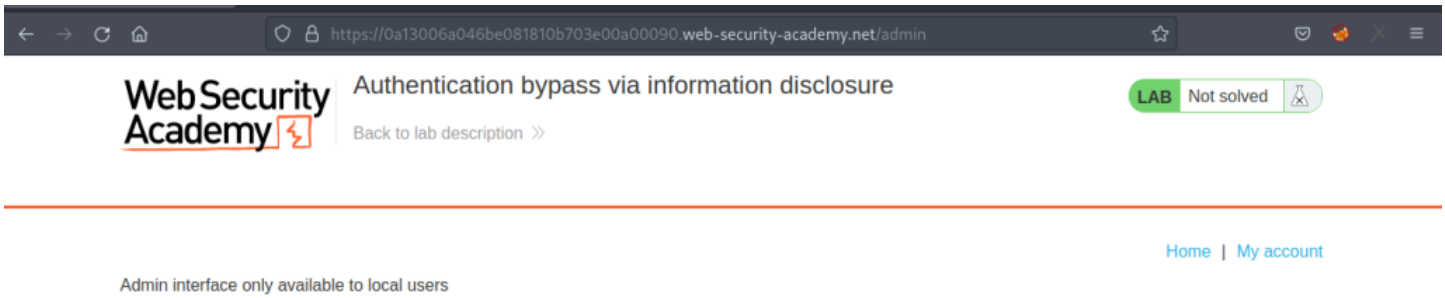
As you can see, we have successfully solved our 3rd lab, Source Code Disclosure via Backup Files. Now we will explore our 4th lab, which is named Authentication Bypass via Information Disclosure.

Authentication bypass via information disclosure

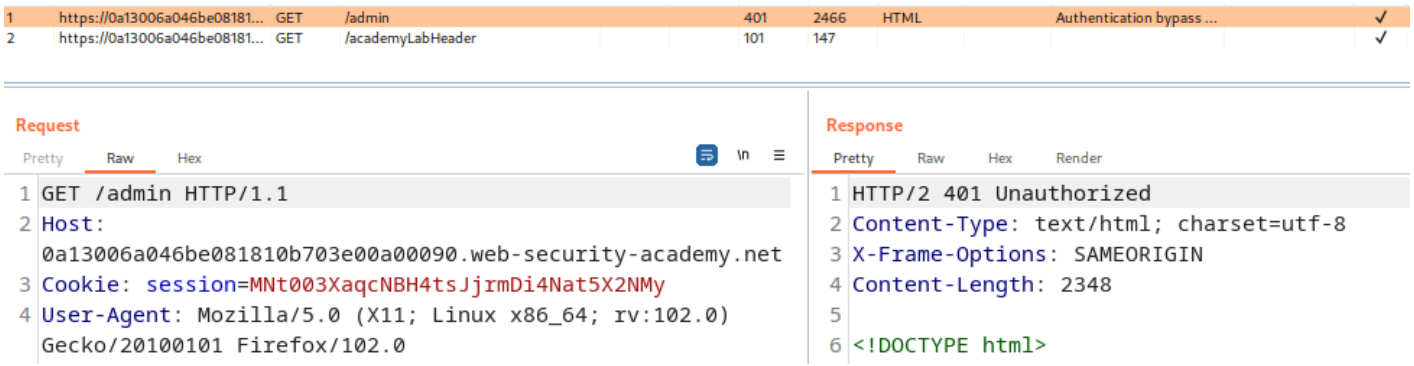


In this lab, we have an authentication bypass vulnerability in the administrator account. We need to exploit this vulnerability and delete Carlos' account as an admin in the lab. We have been given the normal user credentials, which are wiener:peter.

Firstly, we need to access the lab. After accessing the lab, we have been informed that the administrator interface is vulnerable. This means that there is an /admin page in the lab. Let's try to access that page first.



We are unable to access the admin interface because it is only available for local users. To bypass this error message, we need to use a custom header so that we can bypass this error message. To do this, we first need to open Burp Suite and visit the /admin page.



As you can see, it is showing an unauthorized error, which is a 401 error.

To bypass this error, we need a custom header that can help us bypass this security restriction.

```
X-Custom-IP-Authorization: 127.0.0.1
```

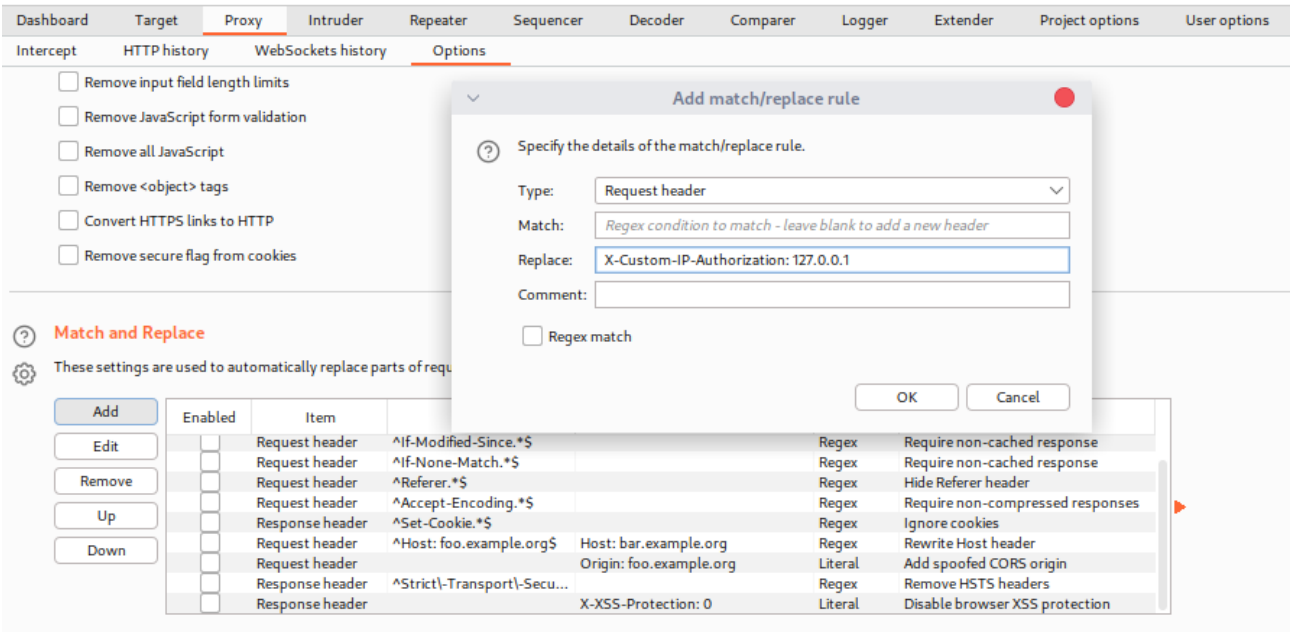
We will send this unauthorized request to the repeater, and then we will add this header there.

```
GET /admin HTTP/2
Host:
0a13006a046be081810b703e00a00090.web-security-academy.net
Cookie: session=MNt003XaqcNBH4tsJjrmDi4Nat5X2NMy
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
Gecko/20100101 Firefox/102.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,ima
ge/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Te: trailers
Connection: close
X-Custom-Ip-Authorization: 127.0.0.1
```

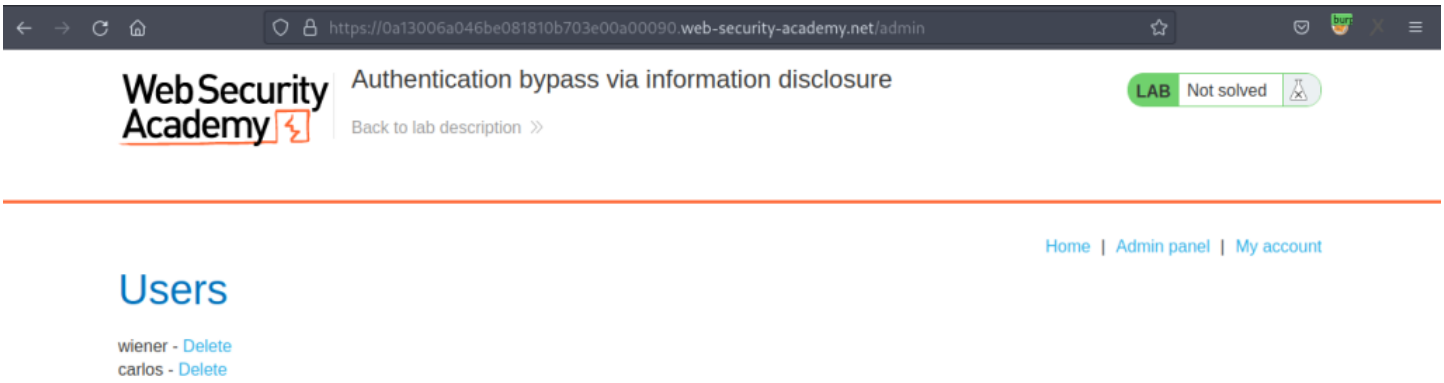
```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: no-cache
X-Frame-Options: SAMEORIGIN
Content-Length: 2883

<!DOCTYPE html>
<html>
  <head>
    <link href=
    /resources/labheader/css/academyLabHeader.css rel=
    stylesheet>
    <link href=/resources/css/labs.css rel=stylesheet>
    <title>
      Authentication bypass via information disclosure
    </title>
  </head>
  <body>
    <script src="/resources/labheader/js/labHeader.js">
```

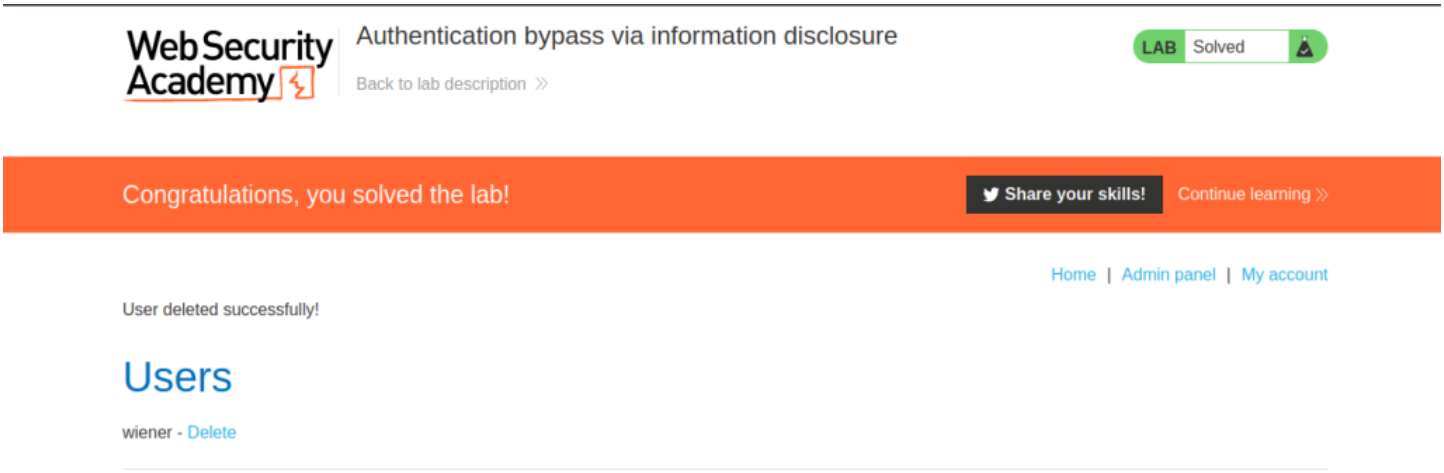
After adding the custom header at the end of the request, we were granted permission as a local user to access the administrator account. However, this is not a permanent solution, so the first step is to go to the proxy tab.



After going to the proxy tab, you should find “match and replace.” Leave the match section empty and insert your custom header in the replace section. Now, you can open the admin interface.

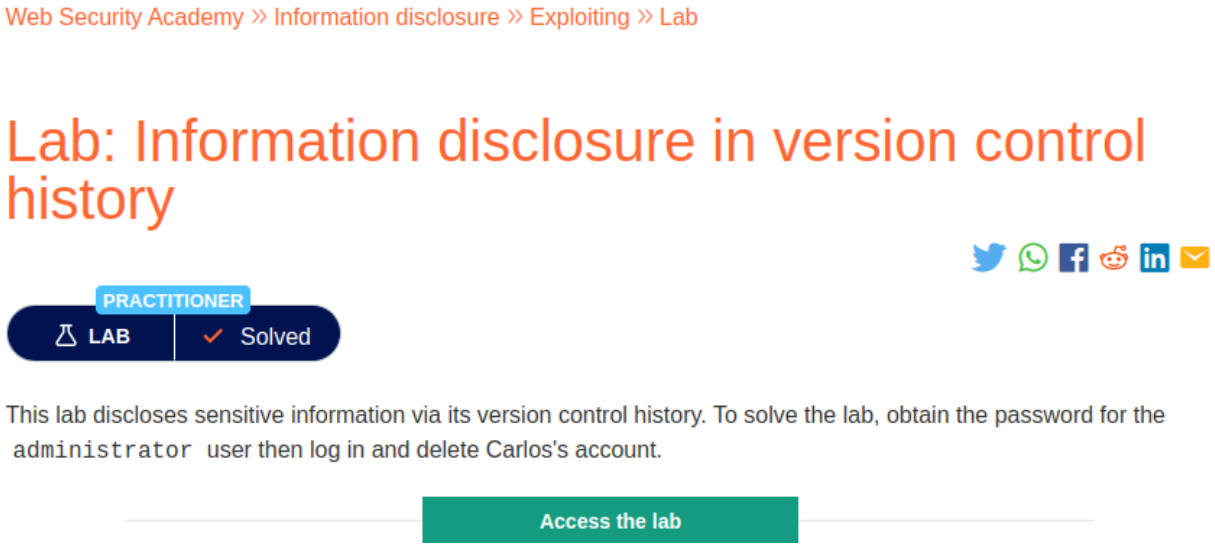


We have accessed the admin interface, and now we need to delete Carlos' account.



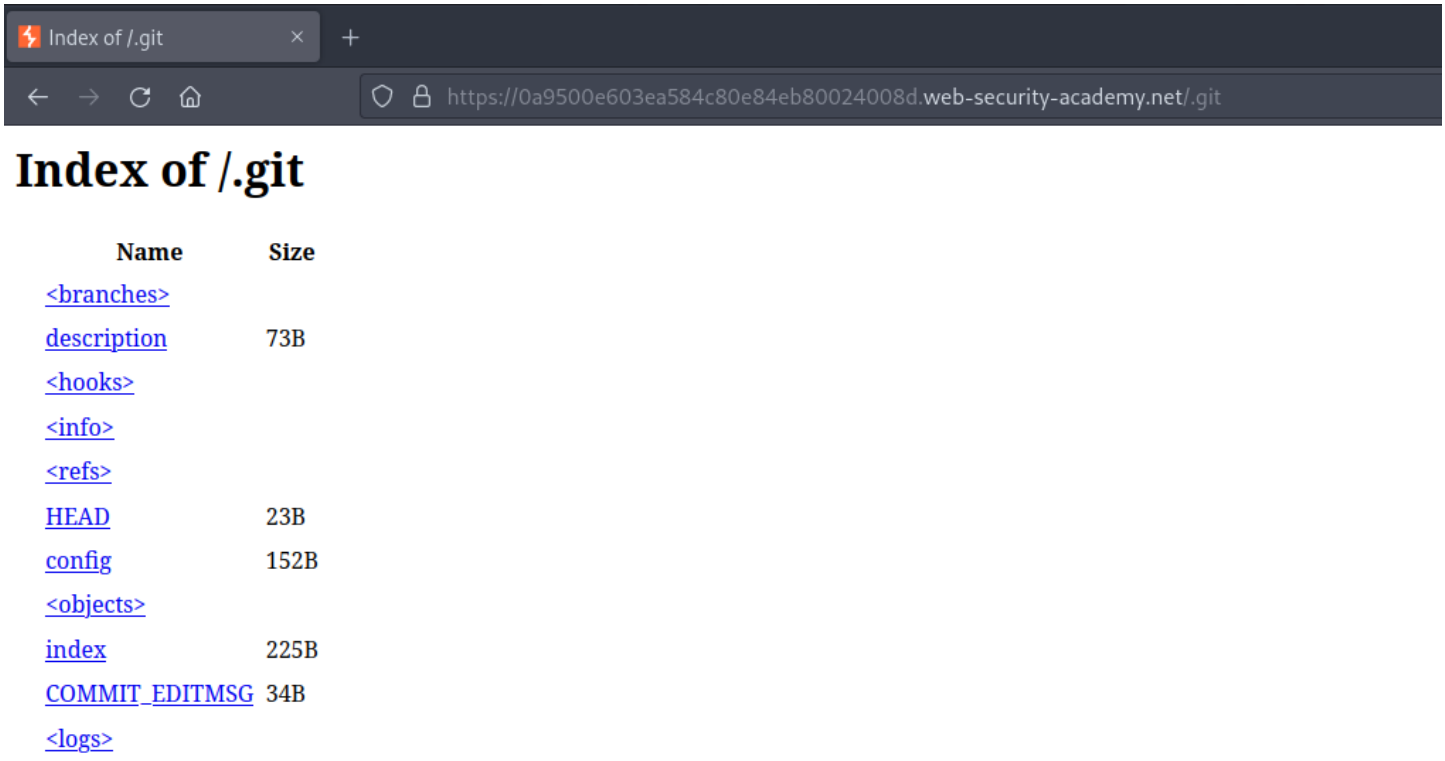
We have successfully deleted Carlos' account, and in the same way, we have solved our 4th lab, which was about authentication bypass via information disclosure.

Information disclosure in version control history



We will now solve the fifth lab, which is about sensitive information disclosure vulnerability via version control history. To solve this lab, we need to obtain the administrator's password and delete the carlos user account.

First, we will access the lab. After accessing the lab, we will try to access the ./git directory.



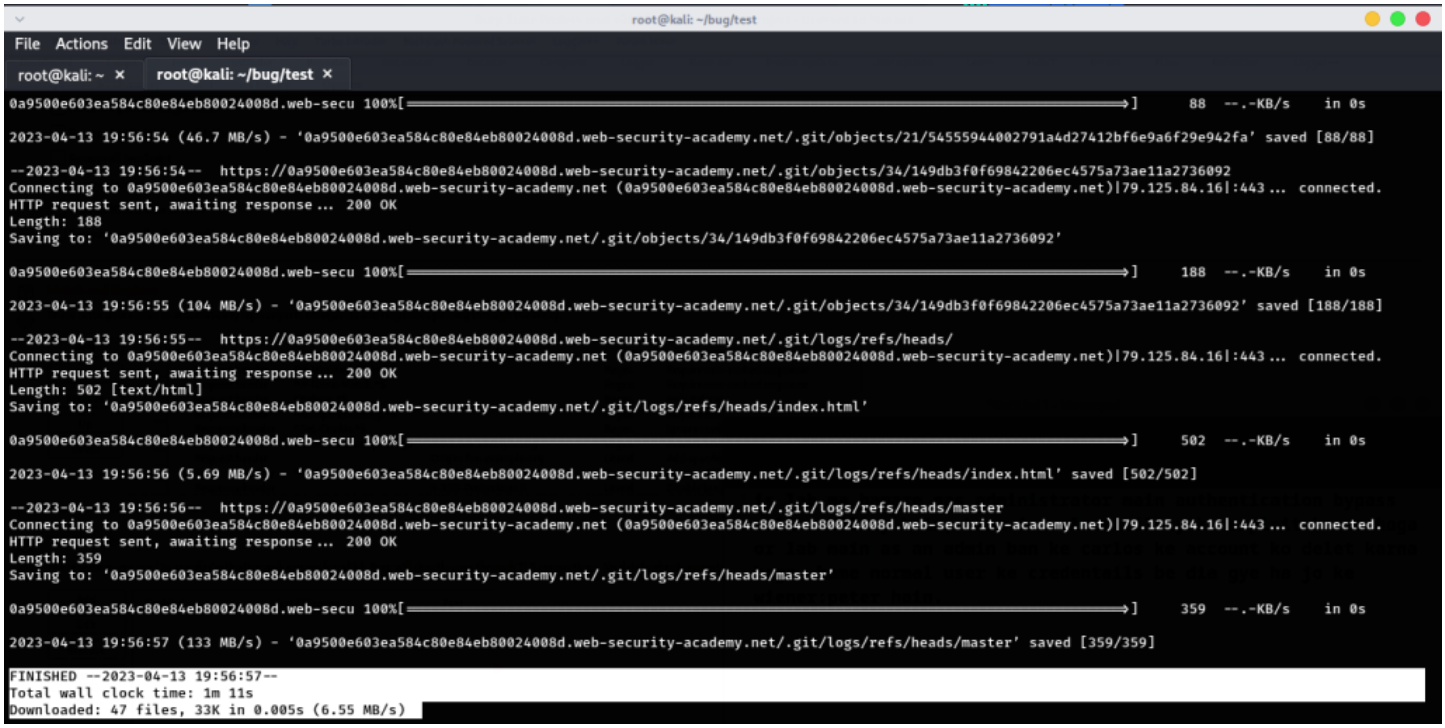
If you are using Linux, you can easily download the entire directory using this command:

```
wget -r https://YOUR-LAB-ID.web-security-academy.net/.git/
```

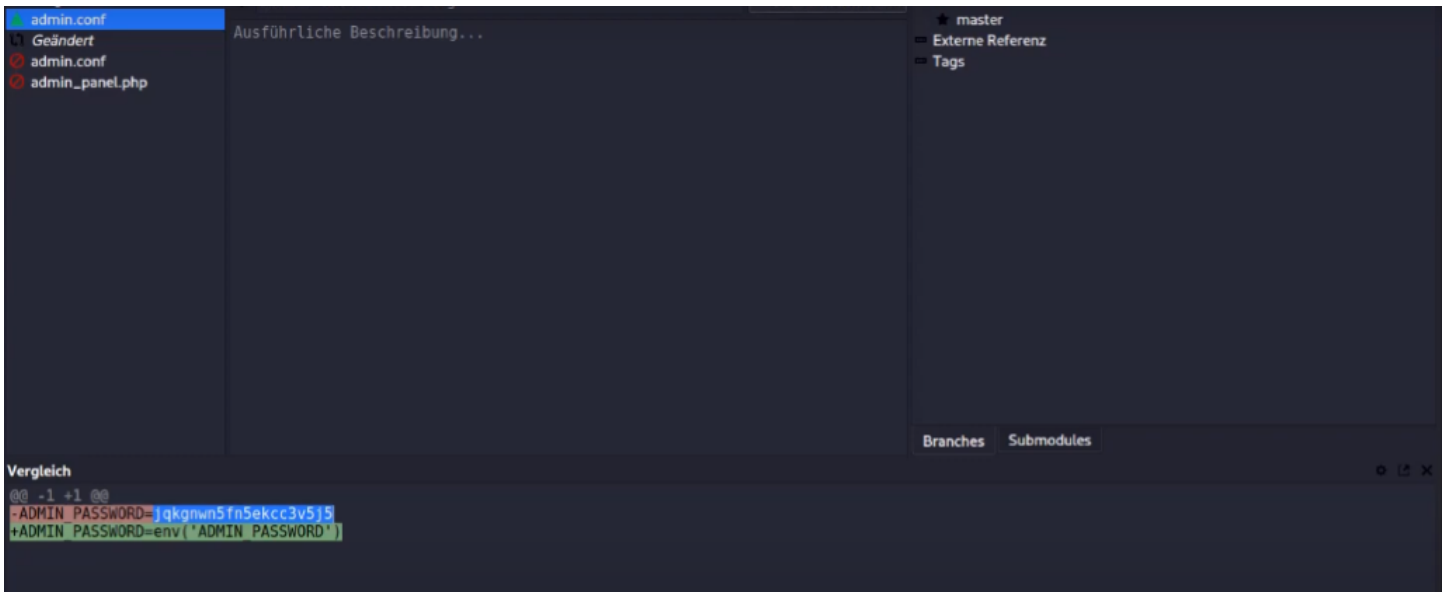
You can set your lab ID in this command. For our lab, we will set it as follows:

```
wget -r https://0a9500e603ea584c80e84eb80024008d.web-security-academy.net/.git/
```

Enter this command in the terminal, and you will have the entire ./git directory downloaded.



I have downloaded all the files, and the administrator's username and password may also be in these files.



After downloading the files, we will use the Git GOLA tool, which is available on Linux. If you don't have it installed, you can install it easily. We will use Git GOLA to open the downloaded folder and then go to the commit and undo the last commit. This way we can reveal the admin credentials.



Using these credentials, we logged in as an administrator and deleted Carlos' account, thus solving the lab.

Automate Information disclosure vulnerabilities



If I wanted to, I could start by automating this entire process to collect data. However, before automating anything, it is important to learn the process manually so that I can understand the automation process well.

In this section, we will automate Information Disclosure vulnerabilities, which will make your work much easier.

To do this, we need a tool called "Feroxbuster".

Installing Feroxbuster is very easy. Just go to your terminal and paste the following command:

```
sudo apt update && sudo apt install -y feroxbuster
```

After doing this, your Feroxbuster will be ready to use. Now, you also need the "seclist" as you will need to mention the files in the list that you want to brute force in the directory.

To use Feroxbuster, the command is:

```
feroxbuster -u https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/ -w comman.txt
```

In my case, I have selected a lab from PortSwigger and I have the wordlist “comman.txt” which is provided by the “seclist“.

```
200      GET        0l        0w        0c https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/analytics
200      GET       16l       29w      435c https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/backup
200      GET      197l      428w    10622c https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/filter
200      GET        2l        4w       32c https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/robots.txt
200      GET      196l      420w    10532c https://0ad600a904d1fb8e80374940001d00a1.web-security-academy.net/
```

result main ap clear dak skty ho ferobuster an hame /backup jasi Important file ko dhond ka da dia ha jo ke is lab ka objective that.

read further : Web Application Penetration Testing Roadmap

Ad fraud Advanced persistent threat (APT) Backdoor Binary planting Botnet Boundary value analysis Broken access control Broken authentication and session management Broken Cryptography Brute force attack Business logic vulnerability Click fraud Clickjacking Code Injection Code obfuscation Command and control (C&C) server Command injection Confidential data CORS (Cross-Origin Resource Sharing) Cross-Site Scripting (XSS) Cross-site scripting (XSS) via other channels (e.g. WebSocket) CSRF (Cross-Site Request Forgery) Data breach Denial-of-service (DoS) attack Directory traversal Distributed denial-of-service (DDoS) attack DNS spoofing DOM-based XSS Dynamic code analysis Eavesdropping Evil twin attack exploit development File inclusion vulnerability File upload vulnerability Firmware vulnerability Forced browsing Fuzz testing HTML injection HTTP response splitting incident response Information leakage Injection attacks Insecure Direct Object References (IDOR) Insecure storage of sensitive data Internet of Things (IoT) security LDAP Injection Local file inclusion (LFI) Logic bomb Malware Man-in-the-Middle (MitM) Attack Mobile application security NoSQL Injection Object injection Open redirect Parameter manipulation Password Reset Poisoning password reuse Path Traversal (PT) penetration testing Personal identifiable information (PII) Pharming Phishing Physical security Privilege escalation Ransomware Reflected XSS Remote code execution (RCE) Remote file inclusion (RFI) reverse engineering Rogue access points Rootkits Sandbox evasion Security misconfiguration Security through obscurity Sensitive Data Exposure Server-side request forgery (SSRF) Server-Side Template Injection (SSTI) Session Fixation Session Hijacking Social Engineering Spear phishing SQL injection (SQLi) Static code analysis Stored XSS TCP/IP hijacking Timing attacks Trojan horse URL manipulation Virus Vulnerability Scanning Watering hole attack Weak Passwords Web cache poisoning Web socket vulnerability Wireless network security XML external entity (XXE) XML injection XPath injection Zero-day vulnerability

Previous Post **Next Post**

You may also like

April 15, 2023



Web Application Penetration Testing Roadmap
April 1, 2023



2-Factor Authentication Simple Bypass
April 8, 2023



Publicly Accessible Localhost Service Free Cloud Service

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

Save my name, email, and website in this browser for the next time I comment.

Post Comment

Search

Our Latest Posts

Fix Error: ChatGPT Too Many Requests in 1 Hour Try Again Later

Roadmap to Becoming a Red Teamer in 2023

Information disclosure vulnerabilities

Web Application Penetration Testing Roadmap

Input Vulnerabilities in Web Applications

5 Essential Tips to Keep Your Facebook ID Secure

OS Command Injection

Broken brute-force protection, multiple credentials per request

2FA bypass using a brute-force attack

Password brute-force via password change

Offline Password Cracking

Prototype Pollution: Client-side prototype pollution in third-party libraries

Prototype Pollution: Client-side prototype pollution via flawed sanitization

Prototype Pollution: DOM XSS via an alternative prototype pollution vector

Prototype Pollution: DOM XSS via client-side prototype pollution

Publicly Accessible Localhost Service Free Cloud Service

Password reset poisoning via middleware

Broken brute-force protection IP block

Username enumeration via account lock

2-Factor Authentication broken logic | Web Security Academy

Education Cyber Security



Copyright © 2022 by **www.novaexperience.net** - All rights reserved.